RKNN2 RK2118 QuickStart

ID: RK-KF-JC-414

Release Version: V2.5.0

Release Date: 2026-09-09

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document describes how to quickly get started using RK2118 NPU SDK.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Log	Reviewer
V2.1.0	Randall	2024-07-30	Support RK2118	Vincent
v2.2.0	HPC	2024-09-04	Update version number	Vincent
v2.3.0	HPC	2024-11-06	Add support for installing RKNN-Toolkit2 via pip source	Vincent
v2.3.2	HPC	2025-04-03	Update version number	Vincent
v2.4.2	HPC	2026-03-19	Update version number	Vincent
v2.5.0	HPC	2026-09-09	1. Update version number 2. Refactor the document structure 3. Update the firmware build method	Vincent

---

Table of Contents

RKNN2 RK2118 QuickStart

1. Introduction

2. Prepare Development Board

2.1 Hardware Checklist

2.2 Introduction to Development Board

2.3 Connect the Development Board to the Computer

3. Prepare Development Environment

3.1 Download RKNN Related Repositories

3.2 Install the RKNN-Toolkit2 Environment on the Computer

3.2.1 Install Python

3.2.1.1 Install miniforge

3.2.1.2 Create Python Environment Using miniforge

3.2.2 Install RKNN-Toolkit2

3.2.3 Check if the RKNN-Toolkit2 Environment is Installed Successfully

3.3 Prepare RK2118 SDK

3.3.1 Repository Download

3.3.2 Compilation environment preparation

3.3.3 RK2118 firmware compilation

3.3.3.1 Enable RKNPU support

3.3.3.2 Compile firmware and package

3.3.3.3 Flashing firmware

4. Run Example Programs

4.1 Introduction to the RKNN SDK

4.2 Model Conversion

4.3 How to Use the NPU Demo

4.3.1 Running Method

4.3.2 Preparing Files on the USB Flash Drive

4.3.3 Viewing the Results

5. RKNN Model Conversion in Docker (Optional)

5.1 Install Docker

5.2 Install the RKNN-Toolkit2 Environment in Docker

5.2.1 Prepare the RKNN-Toolkit2 Image

5.2.2 Query Image Information

5.3 Model Conversion

6. FAQ

6.1 No adb Device Visible

6.2 The role of the rknn_demo_server service

7. Reference Documents

1. Introduction

This document provides a detailed introduction for beginners on how to quickly use RKNN-Toolkit2 on a computer to perform model conversion and deploy it to a Rockchip development board.

Supported platforms: RK2118

2. Prepare Development Board

This chapter will explain how to connect the development board to a computer, divided into three parts:

- Hardware Checklist
- Introduction to Development Board and Connection Tools
- Connect the Development Board to the Computer

2.1 Hardware Checklist

Hardware	Quantity
Computer	1
RK_EVB1_RK2118G_V20 development board	1
USB-A data cable	1
USB-C data cable	1
RK2118 power adapter	1

> **Note:** In the following sections, the RK_EVB1_RK2118G_V20 development board is abbreviated as RK2118.

2.2 Introduction to Development Board

1. Development board

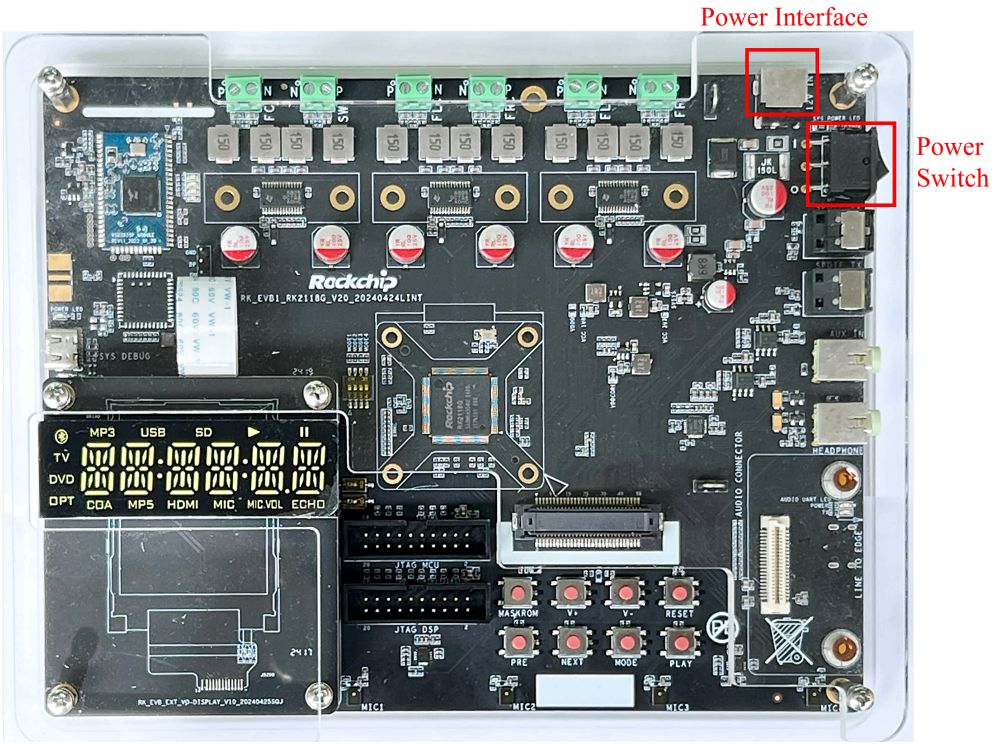


Figure 2-1 RK_EVB1_RK2118G_V20 development board

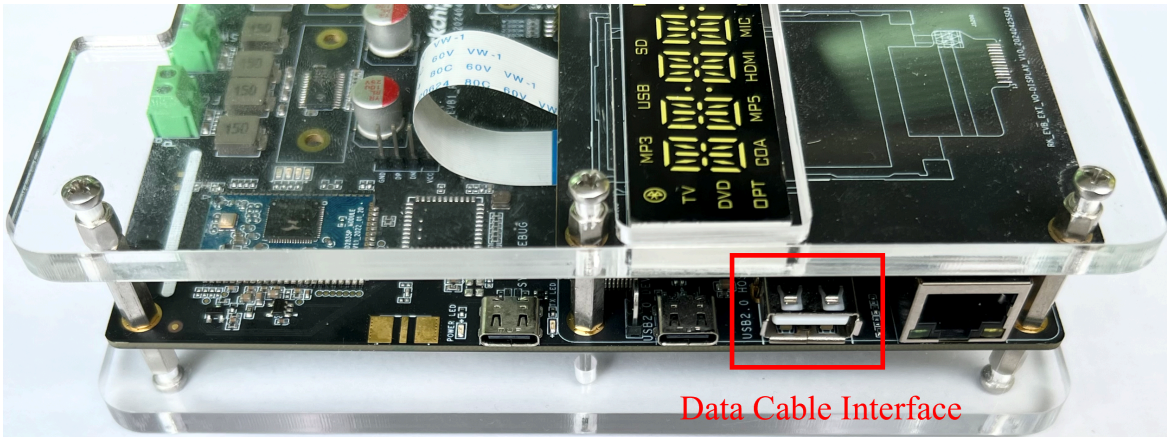


Figure 2-2 RK_EVB1_RK2118G_V20 development board

2. Data cable for connecting development board and computer



Figure2-3 USB-C USB data cable



Figure 2-4 USB-A USB data cable

3. Power adapter



Figure 2-5 Power adapter with 12V-2A output

2.3 Connect the Development Board to the Computer

This section illustrates the connection process using the RK2118 development board as an example.

- Prepare a computer with Ubuntu 18.04, Ubuntu 20.04, or Ubuntu 22.04 operating system.

- Please note that the type and location of the data cable interface on the development board may vary between production batches. Refer to the provided schematic or documentation to locate the power interface and connect the development board's power adapter.
- Activate the power switch and wait for the development board's system to boot up.
- Use a USB Type-C cable to connect the 'SYSTEM DEBUG' port of the development board to the computer.
- Utilize a serial communication tool on the computer to monitor the status of the RK2118. For example, you can use the 'minicom' application to establish a serial connection. The serial port baud rate should be set to 1500000 8N1.

3. Prepare Development Environment

This chapter introduces how to install the development environment directly on the computer. The subsequent sample program running process will also be explained using direct installation as an example. If you need to run the sample program in a Docker environment, you can refer to Chapter 5 to prepare the development environment.

This chapter is divided into three parts:

- Download RKNN Related Repositories
- Install the RKNN-Toolkit2 Environment on the Computer
- Prepare RK2118 SDK

3.1 Download RKNN Related Repositories

It is recommended to create a new directory to store the RKNN repositories. For example, create a folder named "Projects" and place the RKNN-Toolkit2 repositories in that directory. Refer to the following commands:

```
# Create the 'Projects' folder
mkdir Projects

# Switch to this directory
cd Projects

# Download the RKNN-Toolkit2 repository
git clone https://github.com/airockchip/rknn-toolkit2.git --depth 1

# Notice:
# 1. Parameter --depth 1 means to clone only the latest commit
# 2. If the 'git clone' command fails, you can also download the compressed file
directly from GitHub and unzip it locally in this directory.
```

Overall directory structure:

```
Projects
├── rknn-toolkit2
│   ├── doc
│   ├── rknn-toolkit2
│   │   ├── packages
│   │   ├── docker
│   │   └── ...
│   ├── rknpu2
│   │   ├── runtime
│   │   └── ...
│   └── ...
```

3.2 Install the RKNN-Toolkit2 Environment on the Computer

3.2.1 Install Python

If the Python 3.8 environment is not installed on your system, or if there are multiple versions of Python installed, it is recommended to use Miniforge to create a new Python 3.8 environment.

3.2.1.1 Install miniforge

In the terminal window on the computer, execute the following command to check whether miniforge is installed. If already installed, you can skip this step.

```
conda -V
# Reference output: conda 24.3.0, indicating that Miniforge Conda version is 24.3.0
# If it shows 'conda: command not found', it means Miniforge Conda is not
installed.
```

If Miniforge is not installed, you can download the Miniforge installer from the following link:

```
https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-
x86_64.sh
```

Then, install miniforge using the following command:

```
chmod 777 Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh
```

3.2.1.2 Create Python Environment Using miniforge

In the terminal window on the computer, execute the following command to switch to the conda base environment:

```
source /opt/miniforge3/bin/activate
# Upon successful activation, the command prompt will change to the following form:
# (base) xxx@xxx:~$
```

Create a Python 3.8 environment named 'toolkit2' using the following command:

```
conda create -n toolkit2 python=3.8
```

Activate the 'toolkit2' environment. Subsequently, RKNN-Toolkit2 will be installed in this environment.

```
conda activate toolkit2
# (toolkit2) xxx@xxx:~$
```

3.2.2 Install RKNN-Toolkit2

After activating the 'toolkit2' environment, you can install RKNN-Toolkit2 using either the pip source or a local wheel package: - Install via pip source

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# If RKNN-Toolkit2 is already installed, you can upgrade it with the following
command:
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

- Install via local wheel package

```
# Switch to the rknn-toolkit2 directory
cd Projects/rknn-toolkit2/rknn-toolkit2

# Please choose the appropriate requirements file based on the Python version
and processor architecture:
# where cpxx is the Python version
pip install -r packages/x86_64/requirements_cpxx.txt
# pip install -r packages/arm64/arm64_requirements_cpxx.txt

# Install RKNN-Toolkit2
# Please choose the appropriate wheel package file based on the Python version
and processor architecture:
# where x.x.x is the RKNN-Toolkit2 version number, cpxx is the Python version
pip install packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
# pip install packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_aarch64.manylinux2014_aarch64.whl
```

3.2.3 Check if the RKNN-Toolkit2 Environment is Installed Successfully

Execute the following command. if no errors are reported, it indicates that the RKNN-Toolkit2 environment has been successfully installed.

```
# Switch to Python interactive mode
python

# Import the RKNN class
from rknn.api import RKNN
```

If the installation fails, please refer to Chapter 5.4.2, "RKNN-Toolkit2 Installation Issues," in the "RKNN2_SDK_Development_Guide_V2.5.0.pdf" document. It provides detailed solutions for resolving RKNN-Toolkit2 environment installation failures.

3.3 Prepare RK2118 SDK

The example of NPU is included in the RK2118 SDK, so you need to prepare the RK2118 SDK source code first.

3.3.1 Repository Download

- Download repo tool. If repo is already installed, please skip this step.

```
git clone ssh://git@www.rockchip.com.cn/repo/rk/tools/repo -b stable
export PATH=/path/to/repo:$PATH
```

- repository download

```
mkdir rt-thread
cd rt-thread
repo init --repo-url ssh://git@www.rockchip.com.cn/repo/rk/tools/repo -u
ssh://git@www.rockchip.com.cn/rtos/rt-thread/rk/platform/release/manifests -b
master -m rk2118.xml
.repo/repo/repo sync
```

Note: This project requires authorization to access, please contact Rockchip to apply.

3.3.2 Compilation environment preparation

Please refer to `Rockchip_Developer_Guide_RK2118_CN.pdf` in the `rt-thread/Rockchips-docs/manuals/rk2118` directory to build the development environment.

3.3.3 RK2118 firmware compilation

3.3.3.1 Enable RKNPU support

Refer to Chapter 4, "CPU Firmware Compilation," of `Rockchip_Developer_Guide_RK2118_CN.pdf` to compile the firmware and enable RKNPU support. Return to the main directory of the rt-thread project and execute the following command:

```
cd bsp/rockchip/rk2118
cp board/evb1_v20/defconfig_npu_demo .config
scons --menuconfig
```

Please make sure the following options are turned on:

```
RT-Thread Components  --->
  [*] DFS: device virtual file system  --->
       [*]   Using posix-like functions, open/read/write/close
       [*]   Using working directory
       (4)   The maximal number of mounted file system
       (4)   The maximal number of file system type
       (16)  The maximal number of opened files
       [*]   Using mount table for file system
       [*]   Enable elm-chan fatfs
RT-Thread board config  --->
  [*] using DRAM, only available for RK2118B or RK2118G
  [*] Enable hifi4 driver
  HIFI4 Project  --->
    [*] HIFI4 Projects
    (X) npu Demo
  rknn dsp communication config  --->
    [*] Enable rknn dsp demo communication for rk2118
RT-Thread rockchip RK2118 drivers  --->
  Enable RKNPU Driver  --->
    [*] Enable RKNPU Driver
    (X) RKNPU Driver in DSP
```

Notes:

- `board/evb1_v20/defconfig_npu_demo` already includes all of the options listed above, so manual configuration is generally unnecessary. `scons --menuconfig` should only be used for temporary inspection or experimentation.
- For the NPU driver location, select **"RKNPU Driver in DSP"** (`RT_USING_RKNPU_DSP_DRIVER`). In this SDK, the NPU demo runs the RKNN firmware on HIFI4 DSP0 (`components/hifi4/samples/rk2118/npu_demo`), while CPU0 communicates with the DSP through `rknn_demo_server`. The CPU-direct **"RKNPU Driver in RTT"** path is not compatible with this demo.
- After **"npu Demo"** is selected, the RKNN DSP communication components are selected automatically, and the DSP memory layout file `components/hifi4/samples/rk2118/npu_demo/mem_layout.h` is fixed automatically. No manual configuration is required.
- Each time `build.sh` is executed, `.config` is overwritten with the corresponding defconfig. For persistent configuration changes, modify `board/evb1_v20/defconfig_npu_demo` directly.
- The firmware build and packaging command for this board is: `./build.sh board/evb1_v20/package-file npu_demo`

This generates `Image/update.img`, which includes the `dsp0`, `dsp1`, and `dsp2` firmware partitions for `npu_demo`.

3.3.3.2 Compile firmware and package

```
cd bsp/rockchip/rk2118
./build.sh board/evb1_v20/package-file npu_demo
```

This command performs both the build and packaging steps in a single operation:

1. Overwrite `.config` with `board/evb1_v20/defconfig_npu_demo`, then build the CPU0/CPU1 firmware.
2. Invoke `mkimage.sh` to package `dsp0.bin`, `dsp1.bin`, and `dsp2.bin` from `components/hifi4/rtt/dsp_fw/npu_demo/` into the image as DSP firmware partitions (`Type=0x400`).
3. Generate the final image at `Image/update.img`.

Notes:

- If only CPU-side code is modified, you can run `scons -j32` to rebuild the firmware, and then run `./mkimage.sh board/evb1_v20/package-file npu_demo` to package the image.
- If the DSP project is modified (for example, `components/hifi4/samples/rk2118/npu_demo/npu_demo_dsp0`), you must first rebuild the Xtensa project to regenerate `dsp0.bin` under `rtt/dsp_fw/npu_demo/`, and then run the packaging command. Otherwise, the image will still contain the old DSP firmware.
- The DSP firmware partition path is automatically updated by the second argument to `mkimage.sh` (`npu_demo`). Do not modify `package-file` manually.

3.3.3.3 Flashing firmware

Refer to Section 4.1.3.4, "Firmware Burning," of `Rockchip_Developer_Guide_RK2118_CN.pdf` to flash the firmware to the development board. You can also use the upgrade tool included with the SDK to flash `update.img`:

```
upgrade_tool uf Image/update.img
```

4. Run Example Programs

This chapter introduces how to quickly run an example program on the development board. It is divided into three parts:

- Introduction to the RKNN SDK
- RKNN model conversion
- How to use the NPU Demo

4.1 Introduction to the RKNN SDK

RK2118 NPU inference is performed by HIFI4 DSP0; CPU0 does not directly operate the NPU hardware. The related code is divided into three parts:

```
components/hifi4/samples/rk2118/npu_demo/ # DSP0 firmware project
|                                         # (menuconfig: HIFI4 Project → npu Demo)
└── npu_demo_dsp0/
    └── src/rknn_api_test.c               # Main test program on the DSP side; runs
automatically at boot
components/hifi4/rtt/rknn_dsp_comm/       # CPU0-side service (rknn_demo_server)
                                          # Handshakes with the DSP and provides
file I/O
components/hifi4/rtt/dsp_fw/npu_demo/     # DSP firmware for packaging
                                          # (dsp0.bin, updated automatically by the
DSP build)
```

The runtime data flow is as follows:

1. After system startup, CPU0 loads the DSP0 firmware to DSP0 and starts it; `rknn_demo_server` becomes ready on CPU0.
2. `rknn_api_test.c` on DSP0 establishes a handshake with CPU0 through IPC, then requests the model file (`check.rknn`) from the USB flash drive.
3. DSP0 calls APIs such as `rknn_init` and `rknn_run` to perform inference on the NPU.
4. CPU0 writes the inference output back to the USB flash drive for element-by-element comparison with the golden data on the PC.

The CPU0-side application does not call the RKNN APIs directly. All test logic is located in the DSP0 firmware. To modify the test logic, rebuild and package the DSP firmware; see Section 3.3.3.

4.2 Model Conversion

The following uses MobileNetV1 as an example to explain how to convert an RKNN model.

Enter the `rt-thread/components/rknpu/examples/rknn_benchmark/res` directory and run the `convert.py` script. The script converts the original PB model into an RKNN model:

```
cd rt-thread/components/rknpu/examples/rknn_benchmark/res
python convert.py
```

If the conversion succeeds, `mobilenetv1.rknn` and `dog.npy` are generated. `dog.npy` is the model input file.

> **Note:** Before preparing the USB flash drive in the next section, rename `mobilenetv1.rknn` to `check.rknn` and `dog.npy` to `input_0.npy`.

4.3 How to Use the NPU Demo

The following uses `npu_demo` as an example to explain how to use the DSP-based NPU Demo.

4.3.1 Running Method

After the firmware is flashed, DSP0 automatically runs `npu_demo` during system startup; no command needs to be executed manually. Observe the logs through the serial port. All logs have

the `[DSP0 INFO]` prefix.

4.3.2 Preparing Files on the USB Flash Drive

Prepare a FAT32-formatted USB flash drive, place the files in the following directory structure, and insert it into the board's "USB 2.0 HOST" port. The system automatically mounts it at `/udisk`:

```
/udisk
└── golden/
    ├── check.rknn                  # Model file generated in Section 4.2
    └── t0/
        └── <input_tensor_name>.npy # One file for each model input
```

Input file requirements:

- The file must be a standard NumPy `.npy` file with data type **float32**. The number of data bytes must equal the number of input elements × 4. The firmware performs strict validation and reports `Input %d size mismatch` if the size is incorrect.
- The file name must be the model input tensor name, with `/`, `:`, and `.` replaced by `_`. For example, the tensor name `input:0` corresponds to `input_0.npy`.
- Use the file name printed in the serial log during the first run, such as `load file: /udisk/golden/t0/xxx.npy`.
- The input `.npy` file is exported from RKNN-Toolkit2 as golden data.

4.3.3 Viewing the Results

The results are available in two places:

1. **Serial log:** Prints model input/output information, NPU inference time for each iteration, the minimum and maximum values of the output tensor, and the first and last 16 elements. The message `rknn api test done` indicates that the test has completed successfully.
2. **USB flash drive:** The complete output is written back as a float32 file at `/udisk/golden/t0_<output_tensor_name>.npy` (using the same naming rules as above). It can be read directly with NumPy and compared element by element with the golden output on the PC.

The following is an excerpt of the measured MobileNetV1 log:

```
[DSP0 INFO] Read model file success: model_buffer = 0xA016CE80, model_size=...
[DSP0 INFO]   index=0, name=input:0, n_dims=4, dims=[1, 224, 224, 3], ...
[DSP0 INFO]     fmt=NHWC, w_stride=224, size_with_stride=301056, type=FP16
[DSP0 INFO] load file: /udisk/golden/t0/input_0.npy
[DSP0 INFO] Input 0 size check passed: 602112 bytes
[DSP0 INFO] NPU wait 0: Elapse Time = 45ms.
[DSP0 INFO] [0] MobilenetV1/Logits/SpatialSqueeze:0:
[DSP0 INFO] min = -8.1718752, max = 19.21875
```

Notes:

- To replace the model, replace only `check.rknn` and the input `.npy` files under `t0/` on the USB flash drive; rebuilding the firmware is not required.
- To modify the test logic, such as `loop_count` or the dump switch, rebuild and package the DSP firmware; see Section 3.3.3.

5. RKNN Model Conversion in Docker (Optional)

If you need to run the RKNN Python Demo in a Docker environment, please refer to the content in this chapter to prepare the development environment.

Please note that here we provide a Docker image containing the RKNN-Toolkit2 environment, allowing users to directly run RKNN Python Demo without worrying about environment installation issues. However, this Docker image only includes a clean RKNN-Toolkit2 environment and is specifically designed for running RKNN Python Demo.

This chapter is divided into three parts:

- Install Docker
- Install the RKNN-Toolkit2 Environment in Docker
- How to Run RKNN Python Demo

5.1 Install Docker

If Docker is already installed, you can skip this step. If it is not installed, please follow the official documentation for installation.

Docker installation official documentation link: https://docs.docker.com/install/linux/docker-ce/ubuntu/

Note: It is necessary to add the user to the 'docker' group.

```
# Create the docker group
sudo groupadd docker
# Add the current user to the docker group
sudo usermod -aG docker $USER
# Update to activate the docker group
newgrp docker

# Verify that docker commands can be executed without sudo
docker run hello-world
```

The reference output for successful installation is as follows:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

5.2 Install the RKNN-Toolkit2 Environment in Docker

5.2.1 Prepare the RKNN-Toolkit2 Image

This section introduces two methods for creating the RKNN-Toolkit2 image environment. You can choose either method.

1. Create the RKNN Toolkit2 image using a Dockerfile

   In the RKNN-Toolkit2 project, the 'docker/docker_file' folder provides a Dockerfile for building the RKNN-Toolkit2 development environment. Users can create an image using the 'docker build' command, as shown below:

   ```
   # Note: Replace 'xx' and 'x.x.x' with the actual version numbers
   cd Projects/rknn-toolkit2/rknn-toolkit2/docker/docker_file/ubuntu_xx_xx_cpxx
   docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
   ```

2. Create the RKNN Toolkit2 image by loading a pre-packaged Docker image file

   Download the RKNN-Toolkit2 project files for the corresponding version from the following link. After extracting the files, the 'docker/docker_image' folder provides a pre-packaged Docker image with all the development environments.

   Download link from the cloud drive: https://console.zbox.filez.com/l/I00fc3 (Extraction code: rknn)

   Execute the following command to load the corresponding Python version of the image file.

   ```
   # Note: Replace 'x.x.x' with the actual version number of RKNN-Toolkit2, and
   'cpxx' with the Python version
   docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
   ```

5.2.2 Query Image Information

After successfully creating or loading an image, you can view Docker image information using the following command:

```
docker images
```

The corresponding RKNN-Toolkit2 image information is displayed as follows.

```
REPOSITORY        TAG            IMAGE ID        CREATED        SIZE
rknn-toolkit2     x.x.x-cpxx     xxxxxxxxxxxx    1 hours ago    5.89GB
```

5.3 Model Conversion

- Map the files and run the container

Please refer to Chapter 3.1 for downloading the rt-thread project locally. Then, map it into the container and run the container using the 'docker run' command. After running, it will switch to the container's bash environment. Refer to the following commands:

```
# Use the docker run command to create and run the RKNN Toolkit2 container
# Map local files into the container by attaching the -v <host src folder>:<image
dst folder> parameter
docker run -t -i --privileged \
    -v /dev/bus/usb:/dev/bus/usb \
    -v /Projects/rt-thread:/rt-thread \
    rknn-toolkit2:x.x.x-cpxx \
    /bin/bash
```

- Model Conversion

Switch to the rt-thread/components/rknpu/examples/rknn_benchmark/res directory and run the convert.py script.

```
cd rt-thread/components/rknpu/examples/rknn_benchmark/res
python convert.py
```

6. FAQ

6.1 No adb Device Visible

RK2118 currently does not support debugging via adb, so you will not see any adb devices when using this command.

6.2 The role of the rknn_demo_server service

The `rknn_demo_server` service on RK2118 is not used for board debugging with RKNN-Toolkit2. Instead, it provides the communication service between the DSP and the MCU.

7. Reference Documents

- For more detailed usage and interface descriptions of RKNN-Toolkit2 and RKNPU2, please refer to `RKNN2_SDK_Development_Guide_V2.5.0.pdf`.
- For relevant documents of RK2118, please refer to `Rockchip_Developer_Guide_RK2118_CN.pdf`.

> **Note:** For supporting documents whose fixed file names are not specified in this guide, use the actual document names in the current SDK release package.